

CSCI 264 — Assembly Language
Lab 2
Base Conversion of Fractions in C

Last week you converted integers between bases using repeated division. This week you extend that idea to fractions. The digits still come from repeated arithmetic, but now you multiply instead of divide, and the digits come out in the right order the first time.

Warm-up (by hand, on paper)

Show your work, not just the answer.

1. Convert 0.625 to base 2.
2. Convert 11/16 to base 16.

If you can do these by hand, you already know the algorithm your programs have to implement. If you cannot, ask about it in lab before you start coding.

Programming Assignment

Write two C programs, each converting a fraction into another base.

- **Program 1** takes the fraction as a single `double`.
- **Program 2** takes the fraction as two integers: a numerator and a denominator.

Program 1: converting a decimal fraction

Input a decimal fraction, a base, and the number of digits to calculate. Use type `int` for the base `b` and the remainder `r`, and type `double` for the fraction `u`.

Repeat the following statements in a loop, once per output digit:

```
u = u * b;  
r = u;  
u = u - r;
```

The statement `r = u` causes C to do a type conversion with truncation — this truncation is what extracts the next digit. Each resulting value of `r` is one digit of the answer, most significant digit first.

Test cases:

Input	Base	Digits	Output
0.5	2	4	1000
0.625	2	4	1010
0.3	10	3	300

Program 2: converting a numerator/denominator fraction

Input the numerator and denominator of a fraction, a (non-decimal) base, and the number of digits to calculate. Use type `int` for the numerator of the fraction `u`. Your program should work for any base less than 36, using the letters A through Z for digits beyond 9.

Repeat the following statements in a loop, multiplying by the base and dividing by the input denominator `q` to obtain each successive digit:

```
u = u * b;
r = u / q;
u = u % q;
```

Because the digits are calculated left to right, no reversal of the output string is needed — unlike last week's integer conversion.

Test cases:

Numerator	Denominator	Base	Digits	Output
1	2	2	4	1000
3	4	2	3	110
11	16	16	1	B

The third example shows a digit beyond 9 (11, i.e. 'B') produced in base 16.

Output

Store the characters in an array of `char`:

```
char y[100];
```

As each digit `r` is calculated, convert it to its ASCII character and store it in `y`. A remainder is a *number* between 0 and 35. To print it you need the corresponding *character*, and in C a character is just a small integer holding an ASCII code. The two rules you need:

- For digit values 0 through 9, the character code is the value plus 48.
- For digit values 10 through 35, use the letters A through Z: the character code is the value plus 55.

You may also refer to the ASCII table.

Before the terminating zero, store an end-of-line character. That zero is the *null terminator*, and it is how the C library knows where your string stops. Print the finished string with:

```
printf("%s", y);
```

Submission

Submit both source files on Canvas by Tuesday, September 15 and demo both programs to Professor West by the end of the next lab session. Put your name in a comment at the top of each source file. If a classmate helped you put their name in the comment. Disclose any use of Generative AI in a comment at the top.